

Spring Boot JPA + H2 example: Build a CRUD Rest APIs

[Spring Boot JPA + H2 example: Build a CRUD Rest APIs - BezKoder](#)

Last modified: April 4, 2023 [bezcoder](#) [Spring](#)

In this tutorial, we're gonna build a Spring Boot Rest CRUD API example with Maven that use Spring Data JPA to interact with H2 database. You'll know:

- How to configure Spring Data, JPA, Hibernate to work with Database
- How to define Data Models and Repository interfaces
- Way to create Spring Rest Controller to process HTTP requests
- Way to use Spring Data JPA to interact with H2 Database

Contents [\[hide\]](#)

- [Overview of Spring Boot JPA + H2 example](#)
- [Technology](#)
- [Project Structure](#)
- [Create & Setup Spring Boot project](#)
- [Configure Spring Boot, JPA, h2, Hibernate](#)
- [Define Data Model](#)
- [Create Repository Interface](#)
- [Create Spring Rest APIs Controller](#)
- [Run & Test](#)
- [Conclusion](#)
- [Source Code](#)
- [Further Reading](#)

Overview of Spring Boot JPA + H2 example

We will build a Spring Boot Rest Apis using Spring Data JPA with H2 Database for a Tutorial application in that:

- Each Tutorial has id, title, description, published status.
- Apis help to create, retrieve, update, delete Tutorials.
- Apis also support custom finder methods such as find by published status or by title.

These are APIs that we need to provide:

Methods	Urls	Actions
POST	/api/tutorials	create new Tutorial

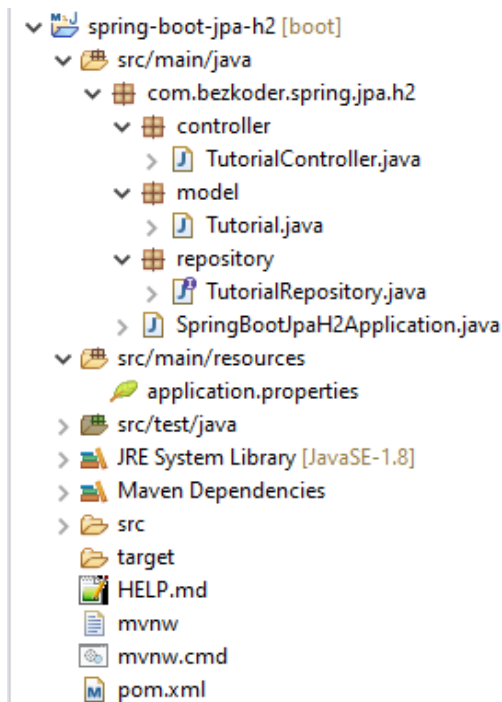
Methods	Urls	Actions
GET	/api/tutorials	retrieve all Tutorials
GET	/api/tutorials/:id	retrieve a Tutorial by :id
PUT	/api/tutorials/:id	update a Tutorial by :id
DELETE	/api/tutorials/:id	delete a Tutorial by :id
DELETE	/api/tutorials	delete all Tutorials
GET	/api/tutorials/published	find all published Tutorials
GET	/api/tutorials?title=[keyword]	find all Tutorials which title contains keyword

- We make CRUD operations & finder methods with Spring Data JPA's `JpaRepository`.
- The database will be H2 Database (in memory or on disk) by configuring project dependency & datasource.

Technology

- Java 8
- Spring Boot 2.7 (with Spring Web MVC, Spring Data JPA)
- H2 Database
- Maven 3.6.1

Project Structure



Let me explain it briefly.

- `Tutorial` data model class corresponds to entity and table *tutorials*.
- `TutorialRepository` is an interface that extends `JpaRepository` for CRUD methods and custom finder methods. It will be autowired in `TutorialController`.
- `TutorialController` is a `RestController` which has request mapping methods for RESTful requests such as: *getAllTutorials, createTutorial, updateTutorial, deleteTutorial, findByPublished...*
- Configuration for Spring Datasource, JPA & Hibernate in **application.properties**.
- **pom.xml** contains dependencies for Spring Boot and H2 Database.

We can improve the example by adding Comments for each Tutorial. It is the One-to-Many Relationship and I write a tutorial for this at:

[Spring Boot One To Many example with JPA, Hibernate](#)

Or add Tags with Many-to-Many Relationship:

[Spring Boot Many to Many example with JPA, Hibernate](#)

Create & Setup Spring Boot project

Use spring boot initializer web site to generate the code of the application. Choose the following dependencies and configurations:

The screenshot shows the Spring Start web application. On the left, there's a sidebar with a hamburger menu. The main area is divided into sections: Project, Language, Spring Boot, Project Metadata, and Dependencies.

Project:

- Gradle - Groovy
- Gradle - Kotlin
- Maven**

Language:

- Java**
- Kotlin
- Groovy

Spring Boot:

- 3.1.0 (SNAPSHOT)
- 3.1.0 (M2)
- 3.0.6 (SNAPSHOT)
- 3.0.5**
- 2.7.11 (SNAPSHOT)
- 2.7.10

Project Metadata:

- Group: rest
- Artifact: crud
- Name: crud
- Description: CRUD project for Spring Boot
- Package name: rest.crud
- Packaging: **Jar**, War
- Java: 20, **17**, 11, 8

Dependencies:

- Spring Web** (WEB): Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.
- Spring Data JPA** (SQL): Persist data in SQL stores with Java Persistence API using Spring Data and Hibernate.
- H2 Database** (SQL): Provides a fast in-memory database that supports JDBC API and R2DBC access, with a small (2mb) footprint. Supports embedded and server modes as well as a browser based console application.
- Rest Repositories** (WEB): Exposing Spring Data repositories over REST via Spring Data REST.

At the top right of the Dependencies section, there is a button: **ADD DEPENDENCIES... CTRL + B**

Use [Spring web tool](#) or your development tool ([Spring Tool Suite](#), Eclipse, [IntelliJ](#)) to create a Spring Boot project.

Then open **pom.xml** and add these dependencies:

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>

<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
</dependency>

<dependency>
    <groupId>com.h2database</groupId>
    <artifactId>h2</artifactId>
    <scope>runtime</scope>
</dependency>
```

Configure Spring Boot, JPA, h2, Hibernate

Under **src/main/resources** folder, open *application.properties* and write these lines.

```
spring.datasource.url=jdbc:h2:mem:testdb
```

```

spring.datasource.driverClassName=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=

spring.jpa.show-sql=true
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.H2Dialect
spring.jpa.hibernate.ddl-auto= update

spring.h2.console.enabled=true
# default path: h2-console
spring.h2.console.path=/h2-ui

#server.port=8888

```

- `spring.datasource.url: jdbc:h2:mem:[database-name]` for In-memory database and `jdbc:h2:file:[path/database-name]` for disk-based database.
- `spring.datasource.username` & `spring.datasource.password` properties are the same as your database installation.
- Spring Boot uses Hibernate for JPA implementation, we configure `H2Dialect` for H2 Database
- `spring.jpa.hibernate.ddl-auto` is used for database initialization. We set the value to `update` value so that a table will be created in the database automatically corresponding to defined data model. Any change to the model will also trigger an update to the table. For production, this property should be `validate`.
- `spring.h2.console.enabled=true` tells the Spring to start H2 Database administration tool and you can access this tool on the browser: `http://localhost:8080/h2-console`.
- `spring.h2.console.path=/h2-ui` is for H2 console's url, so the default url `http://localhost:8080/h2-console` will change to `http://localhost:8080/h2-ui`.

Define Data Model

Our Data model is Tutorial with four fields: id, title, description, published. In **model** package, we define `Tutorial` class.

model/Tutorial.java

```

package com.bezkoder.spring.jpa.h2.model;

import javax.persistence.*;
// import jakarta.persistence.*; // for Spring Boot 3

@Entity
@Table(name = "tutorials")
public class Tutorial {

```

```
@Id
@GeneratedValue(strategy = GenerationType.AUTO)
private long id;
@Column(name = "title")
private String title;
@Column(name = "description")
private String description;
@Column(name = "published")
private boolean published;
public Tutorial() {
}
public Tutorial(String title, String description, boolean published) {
    this.title = title;
    this.description = description;
    this.published = published;
}
public long getId() {
    return id;
}

public String getTitle() {
    return title;
}
public void setTitle(String title) {
    this.title = title;
}
public String getDescription() {
    return description;
}
public void setDescription(String description) {
    this.description = description;
}
public boolean isPublished() {
    return published;
}
public void setPublished(boolean isPublished) {
```

```

        this.published = isPublished;
    }
    @Override
    public String toString() {
        return "Tutorial [id=" + id + ", title=" + title + ", desc=" +
description + ", published=" + published + "]";
    }
}

```

- `@Entity` annotation indicates that the class is a persistent Java class.
- `@Table` annotation provides the table that maps this entity.
- `@Id` annotation is for the primary key.
- `@GeneratedValue` annotation is used to define generation strategy for the primary key. `GenerationType.AUTO` means Auto Increment field.
- `@Column` annotation is used to define the column in database that maps annotated field.

Create Repository Interface

Let's create a repository to interact with Tutorials from the database.
In **repository** package, create `TutorialRepository` interface that extends `JpaRepository`.

repository/TutorialRepository.java

```

package com.bezkoder.spring.jpa.h2.repository;
import java.util.List;
import org.springframework.data.jpa.repository.JpaRepository;
import com.bezkoder.spring.jpa.h2.model.Tutorial;

public interface TutorialRepository extends JpaRepository<Tutorial, Long> {
    List<Tutorial> findByPublished(boolean published);

    List<Tutorial> findByTitleContaining(String title);
}

```

Now we can use `JpaRepository`'s methods: `save()`, `findOne()`, `findById()`, `findAll()`, `count()`, `delete()`, `deleteById()`... without implementing these methods.

We also define custom finder methods:

- `findByPublished()`: returns all Tutorials with `published` having value as input `published`.
- `findByTitleContaining()`: returns all Tutorials which title contains input `title`.

The implementation is plugged in by [Spring Data JPA](#) automatically.

More Derived queries at:

[JPA Repository query example in Spring Boot](#)

Custom query with `@Query` annotation:

[Spring JPA @Query example: Custom query in Spring Boot](#)

You can modify this Repository:

- to work with Pagination, the instruction can be found at:
[Spring Boot Pagination & Filter example | Spring JPA, Pageable](#)
- or to sort/order by multiple fields with the tutorial:
[Spring Data JPA Sort/Order by multiple Columns | Spring Boot](#)

You also find way to write Unit Test for this JPA Repository at:
[Spring Boot Unit Test for JPA Repository with @DataJpaTest](#)

Create Spring Rest APIs Controller

Finally, we create a controller that provides APIs for creating, retrieving, updating, deleting and finding Tutorials.

controller/TutorialController.java

```
package com.bezkoder.spring.jpa.h2.controller;

import java.util.ArrayList;
import java.util.List;
import java.util.Optional;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.CrossOrigin;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;

import com.bezkoder.spring.jpa.h2.model.Tutorial;
import com.bezkoder.spring.jpa.h2.repository.TutorialRepository;

@CrossOrigin(origins = "http://localhost:8081")
@RestController
@RequestMapping("/api")
```



```

public class TutorialController {
    @Autowired
    TutorialRepository tutorialRepository;

    @GetMapping("/tutorials")
    public ResponseEntity<List<Tutorial>> getAllTutorials(@RequestParam(required
= false) String title) {
        try {
            List<Tutorial> tutorials = new ArrayList<Tutorial>();
            if (title == null)

tutorialRepository.findAll().forEach(tutorials::add);
            else

tutorialRepository.findByTitleContaining(title).forEach(tutorials::add);
            if (tutorials.isEmpty()) {
                return new ResponseEntity<>(HttpStatus.NO_CONTENT);
            }

            return new ResponseEntity<>(tutorials, HttpStatus.OK);
        } catch (Exception e) {
            return new ResponseEntity<>(null,
HttpStatus.INTERNAL_SERVER_ERROR);
        }
    }
    @GetMapping("/tutorials/{id}")
    public ResponseEntity<Tutorial> getTutorialById(@PathVariable("id") long id)
{
        Optional<Tutorial> tutorialData = tutorialRepository.findById(id);

        if (tutorialData.isPresent()) {
            return new ResponseEntity<>(tutorialData.get(),
HttpStatus.OK);
        } else {
            return new ResponseEntity<>(HttpStatus.NOT_FOUND);
        }
    }
    @PostMapping("/tutorials")

```

```

    public ResponseEntity<Tutorial> createTutorial(@RequestBody Tutorial
tutorial) {
        try {
            Tutorial _tutorial = tutorialRepository
                .save(new Tutorial(tutorial.getTitle(),
tutorial.getDescription(), false));
            return new ResponseEntity<>(_tutorial, HttpStatus.CREATED);
        } catch (Exception e) {
            return new ResponseEntity<>(null,
HttpStatus.INTERNAL_SERVER_ERROR);
        }
    }
    @PostMapping("/tutorials/{id}")
    public ResponseEntity<Tutorial> updateTutorial(@PathVariable("id") long id,
@RequestBody Tutorial tutorial) {
        Optional<Tutorial> tutorialData = tutorialRepository.findById(id);

        if (tutorialData.isPresent()) {
            Tutorial _tutorial = tutorialData.get();
            _tutorial.setTitle(tutorial.getTitle());
            _tutorial.setDescription(tutorial.getDescription());
            _tutorial.setPublished(tutorial.isPublished());
            return new
ResponseEntity<>(tutorialRepository.save(_tutorial), HttpStatus.OK);
        } else {
            return new ResponseEntity<>(HttpStatus.NOT_FOUND);
        }
    }
    @DeleteMapping("/tutorials/{id}")
    public ResponseEntity<HttpStatus> deleteTutorial(@PathVariable("id") long
id) {
        try {
            tutorialRepository.deleteById(id);
            return new ResponseEntity<>(HttpStatus.NO_CONTENT);
        } catch (Exception e) {
            return new
ResponseEntity<>(HttpStatus.INTERNAL_SERVER_ERROR);
        }
    }
}

```

```

@DeleteMapping("/tutorials")
public ResponseEntity<HttpStatus> deleteAllTutorials() {
    try {
        tutorialRepository.deleteAll();
        return new ResponseEntity<>(HttpStatus.NO_CONTENT);
    } catch (Exception e) {
        return new
ResponseEntity<>(HttpStatus.INTERNAL_SERVER_ERROR);
    }
}

@GetMapping("/tutorials/published")
public ResponseEntity<List<Tutorial>> findByPublished() {
    try {
        List<Tutorial> tutorials =
tutorialRepository.findByPublished(true);

        if (tutorials.isEmpty()) {
            return new ResponseEntity<>(HttpStatus.NO_CONTENT);
        }
        return new ResponseEntity<>(tutorials, HttpStatus.OK);
    } catch (Exception e) {
        return new
ResponseEntity<>(HttpStatus.INTERNAL_SERVER_ERROR);
    }
}
}

```

- @CrossOrigin is for configuring allowed origins.
- @RestController annotation is used to define a controller and to indicate that the return value of the methods should be bound to the web response body.
- @RequestMapping("/api") declares that all Apis' url in the controller will start with /api.
- We use @Autowired to inject TutorialRepository bean to local variable.

You can find way to write Unit test for this Controller at:

[Spring Boot Unit Test for Rest Controller](#)

Run & Test

Run Spring Boot application with command: `mvn spring-boot:run`.

tutorials table will be automatically generated in Database.

Let's open H2 console with url: `http://localhost:8080/h2-ui`:

- For In-memory database:

← → ↻ 🏠 ⓘ localhost:8080/h2-ui/test.do?jsessionid=b99e8edea5b7ac7113e6a7e9d1889e72

English ▼ Preferences Tools Help

Login

Saved Settings: Generic H2 (Embedded) ▼

Setting Name: Generic H2 (Embedded) Save Remove

Driver Class: org.h2.Driver

JDBC URL: jdbc:h2:mem:testdb

User Name: sa

Password:

Connect Test Connection

– For on Disk database:

← → ↻ 🏠 ⓘ localhost:8080/h2-ui/login.jsp?jsessionid=b99e8edea5b7ac7113e6a7e9d1889e72

English ▼ Preferences Tools Help

Login

Saved Settings: Generic H2 (Embedded) ▼

Setting Name: Generic H2 (Embedded) Save Remove

Driver Class: org.h2.Driver

JDBC URL: jdbc:h2:file:~/h2/testdb

User Name: sa

Password:

Connect Test Connection

Click on **Connect** button, then check H2 database, you can see things like this:

jdbc:h2:mem:testdb

TUTORIALS

- ID
- DESCRIPTION
- PUBLISHED
- TITLE
- Indexes

INFORMATION_SCHEMA

Sequences

Users

H2 1.4.200 (2019-10-14)

Run Run Selected Auto complete Clear SQL statement:

SELECT * FROM TUTORIALS TUTORIALS|

SELECT * FROM TUTORIALS TUTORIALS;
ID DESCRIPTION PUBLISHED TITLE
(no rows, 3 ms)

Edit

Create some Tutorials:

POST http://localhost:8080/api/tutorials Send

Params Auth Headers (10) Body Pre-req. Tests Settings

raw JSON

```

1 {
2   "title": "Spring Boot JPA H2 example Tut#5",
3   "description": "Tut#5 Description"
4 }

```

Body 201 Created 35 ms 361 B

Pretty Raw Preview Visualize JSON

```

1 {
2   "id": 5,
3   "title": "Spring Boot JPA H2 example Tut#5",
4   "description": "Tut#5 Description",
5   "published": false
6 }

```

H2 database tutorials table after that:

Run

Run Selected

Auto complete

Clear

SQL statement:

SELECT * FROM TUTORIALS TUTORIALS

SELECT * FROM TUTORIALS TUTORIALS;

ID	DESCRIPTION	PUBLISHED	TITLE
1	Tut#1 Description	FALSE	Spring Boot Tut#1
2	Tut#2 Description	FALSE	Spring Data JPA Tut#2
3	Tut#3 Description	FALSE	H2 Database Tut#3
4	Tut#4 Description	FALSE	Spring Boot CRUD Tut#4
5	Tut#5 Description	FALSE	Spring Boot JPA H2 example Tut#5

(5 rows, 10 ms)

Update some Tutorials:

PUT

▼

http://localhost:8080/api/tutorials/3

Send

Params Auth Headers (10) Body ● Pre-req. Tests Settings

raw ▼ JSON ▼

1 {

2 "title": "H2 Database Tut#3",

3 "description": "Desc for Tut#3",

4 "published": true

5 }

Body ▼  200 OK 93 ms 337 B

Pretty Raw Preview Visualize JSON ▼ 

1 {

2 "id": 3,

3 "title": "H2 Database Tut#3",

4 "description": "Desc for Tut#3",

5 "published": true

6 }

The table data is changed:

Run

Run Selected

Auto complete

Clear

SQL statement:

SELECT * FROM TUTORIALS TUTORIALS|

SELECT * FROM TUTORIALS TUTORIALS;

ID	DESCRIPTION	PUBLISHED	TITLE
1	Desc for Tut#1	TRUE	Spring Boot Tut#1
2	Tut#2 Description	FALSE	Spring Data JPA Tut#2
3	Desc for Tut#3	TRUE	H2 Database Tut#3
4	Tut#4 Description	FALSE	Spring Boot CRUD Tut#4
5	Desc for Tut#5	TRUE	Spring Boot JPA H2 example Tut#5

(5 rows, 2 ms)

Retrieve all Tutorials:

GET

http://localhost:8080/api/tutorials

Send

ParamsAuthHeaders (7)BodyPre-req. TestsSettings

Body

200 OK53 ms711 B

PrettyRawPreviewVisualizeJSON

```
1  [
2    {
3      "id": 1,
4      "title": "Spring Boot Tut#1",
5      "description": "Desc for Tut#1",
6      "published": true
7    },
8    {
9      "id": 2,
10     "title": "Spring Data JPA Tut#2",
11     "description": "Tut#2 Description",
12     "published": false
13   },
14   {
15     "id": 3,
16     "title": "H2 Database Tut#3",
17     "description": "Desc for Tut#3",
18     "published": true
19   },
20   {
21     "id": 4,
22     "title": "Spring Boot CRUD Tut#4",
23     "description": "Tut#4 Description",
24     "published": false
25   },
26   {
27     "id": 5,
28     "title": "Spring Boot JPA H2 example Tut#5",
29     "description": "Desc for Tut#5",
30     "published": true
31   }
32 ]
```

Retrieve a Tutorial by Id:

GET

▼

http://localhost:8080/api/tutorials/5

Send

ParamsAuthHeaders (7)BodyPre-req. TestsSettings

Body ▼

200 OK16 ms352 B

PrettyRawPreviewVisualize

JSON ▼

1

{

2

"id": 5,

3

"title": "Spring Boot JPA H2 example Tut#5",

4

"description": "Desc for Tut#5",

5

"published": true

6

}

Find all **published** Tutorials:

GET

http://localhost:8080/api/tutorials/published

Send

ParamsAuthHeaders (7)BodyPre-req. TestsSettings

Body

200 OK59 ms524 B

PrettyRawPreviewVisualizeJSON

```
1  [
2    {
3      "id": 1,
4      "title": "Spring Boot Tut#1",
5      "description": "Desc for Tut#1",
6      "published": true
7    },
8    {
9      "id": 3,
10     "title": "H2 Database Tut#3",
11     "description": "Desc for Tut#3",
12     "published": true
13   },
14   {
15     "id": 5,
16     "title": "Spring Boot JPA H2 example Tut#5",
17     "description": "Desc for Tut#5",
18     "published": true
19   }
20 ]
```

Find all Tutorials which title contains string 'ot':

GET

http://localhost:8080/api/tutorials?title=ot

Send

Params ● Auth Headers (7) Body Pre-req. Tests Settings

Body ▼

200 OK

13 ms

533 B

PrettyRawPreviewVisualize

JSON ▼



1

[

2

{

3

"id": 1,

4

"title": "Spring Boot Tut#1",

5

"description": "Desc for Tut#1",

6

"published": true

7

},

8

{

9

"id": 4,

10

"title": "Spring Boot CRUD Tut#4",

11

"description": "Tut#4 Description",

12

"published": false

13

},

14

{

15

"id": 5,

16

"title": "Spring Boot JPA H2 example Tut#5",

17

"description": "Desc for Tut#5",

18

"published": true

19

}

20

]

Delete a Tutorial:

DELETE

http://localhost:8080/api/tutorials/4

Send

Params Auth Headers (7) Body Pre-req. Tests Settings

Body ▼

204 No Content

55 ms

201 B

PrettyRawPreviewVisualize

Text ▼



1

Run Run Selected Auto complete Clear SQL statement:

SELECT * FROM TUTORIALS TUTORIALS|

SELECT * FROM TUTORIALS TUTORIALS;

ID	DESCRIPTION	PUBLISHED	TITLE
1	Desc for Tut#1	TRUE	Spring Boot Tut#1
2	Tut#2 Description	FALSE	Spring Data JPA Tut#2
3	Desc for Tut#3	TRUE	H2 Database Tut#3
5	Desc for Tut#5	TRUE	Spring Boot JPA H2 example Tut#5

(4 rows, 2 ms)

Delete all Tutorials:

DELETE ▼ http://localhost:8080/api/tutorials Send

Params Auth Headers (7) Body Pre-req. Tests Settings

Body ▼ 204 No Content 21 ms 201 B

Pretty Raw Preview Visualize Text ▼ ↺

1

H2 database table is clean now:

Run Run Selected Auto complete Clear SQL statement:

SELECT * FROM TUTORIALS TUTORIALS|

SELECT * FROM TUTORIALS TUTORIALS;

ID	DESCRIPTION	PUBLISHED	TITLE
----	-------------	-----------	-------

(no rows, 2 ms)

You can also test this Spring Boot App with Client in one of these posts:

- [Simple HTTP Client using Axios](#)
- [Simple HTTP Client using Fetch API](#)
- [Angular 8 CRUD Application example with Web API](#)
- [Angular 10 CRUD Application example with Web API](#)

- [Angular 11 CRUD Application example with Web API](#)
- [Angular 12 CRUD Application example with Web API](#)
- [Angular 13 CRUD Application example with Web API](#)
- [Angular 14 CRUD Application example with Web API](#)
- [Angular 15 CRUD Application example with Web API](#)
- [Vue 2 CRUD Application with Vue Router & Axios](#)
- [Vue 3 CRUD Application with Axios & Vue Router](#)
- [React CRUD example with Axios](#)
- [React Redux CRUD example](#)

Conclusion

Today we've built a Rest CRUD API using Spring Boot, Spring Data JPA working with H2 Database example.

We also see that `JpaRepository` supports a great way to make CRUD operations and custom finder methods without need of boilerplate code.

Custom query with `@Query` annotation:

[Spring JPA @Query example: Custom query in Spring Boot](#)

If you want to add Pagination to this Spring project, you can find the instruction at:

[Spring Boot Pagination & Filter example | Spring JPA, Pageable](#)

To sort/order by multiple fields:

[Spring Data JPA Sort/Order by multiple Columns | Spring Boot](#)

Handle Exception for this Rest APIs is necessary:

- [Spring Boot @ControllerAdvice & @ExceptionHandler example](#)
- [@RestControllerAdvice example in Spring Boot](#)

Or way to write Unit Test:

- [Spring Boot Unit Test for JPA Repository](#)
- [Spring Boot Unit Test for Rest Controller](#)

Happy learning! See you again.

Further Reading

- [Secure Spring Boot App with Spring Security & JWT Authentication](#)
- [Spring Data JPA Reference Documentation](#)

Fullstack CRUD App:

- [Spring Boot Thymeleaf example](#)
- [Vue + Spring Boot example](#)
- [Angular 8 + Spring Boot example](#)
- [Angular 10 + Spring Boot example](#)
- [Angular 11 + Spring Boot example](#)
- [Angular 12 + Spring Boot example](#)
- [Angular 13 + Spring Boot example](#)

- [Angular 14 + Spring Boot example](#)
- [Angular 15 + Spring Boot example](#)
- [React + Spring Boot example](#)

More Practice:

- [Secure Spring Boot App with Spring Security & JWT Authentication](#)
- [Spring Boot Rest XML example – Web service with XML Response](#)
- [Spring Boot Multipart File upload example](#)
- [Spring Boot Pagination and Sorting example](#)

We can improve the example by adding Comments for each Tutorial. It is the One-to-Many Relationship and I write a tutorial for this at:

[Spring Boot One To Many example with JPA, Hibernate](#)

Or add Tags with Many-to-Many Relationship:

[Spring Boot Many to Many example with JPA, Hibernate](#)

Source Code

You can find the complete source code for this tutorial on [Github](#).

More Derived queries at:

[JPA Repository query example in Spring Boot](#)

Using other databases:

- [Spring JPA + SQL Server](#)
- [Spring JPA + PostgreSQL](#)
- [Spring JPA + MySQL](#)
- [Spring Data + MongoDB](#)

Or JdbcTemplate instead:

[Spring Boot JdbcTemplate example: CRUD Rest API](#)

Reactive with R2DBC:

[Spring Boot R2DBC + H2 example](#)

Documentation: [Spring Boot + Swagger 3 example \(with OpenAPI 3\)](#)

Caching: [Spring Boot Redis Cache example](#)

[crud](#) [h2 database](#) [hibernate](#) [jpa repository](#) [rest api](#) [spring boot](#) [spring data jpa](#)

Fullstack:

- [Spring Boot Thymeleaf example](#)
- [Vue + Spring Boot example](#)
- [Angular 8 + Spring Boot example](#)
- [Angular 10 + Spring Boot example](#)
- [Angular 11 + Spring Boot example](#)
- [Angular 12 + Spring Boot example](#)
- [Angular 13 + Spring Boot example](#)
- [Angular 14 + Spring Boot example](#)
- [Angular 15 + Spring Boot example](#)
- [React + Spring Boot example](#)

JdbcTemplate instead:

[Spring Boot JdbcTemplate example: CRUD Rest API](#)

Reactive with R2DBC:

[Spring Boot R2DBC + H2 example](#)

Exception Handling:

- [Spring Boot @ControllerAdvice & @ExceptionHandler example](#)
- [@RestControllerAdvice example in Spring Boot](#)

Testing:

- [Spring Boot Unit Test for JPA Repository](#)
- [Spring Boot Unit Test for Rest Controller](#)

Associations:

- [Spring Boot One To One example with JPA, Hibernate](#)
- [Spring Boot One To Many example with JPA, Hibernate](#)
- [Spring Boot Many to Many example with JPA, Hibernate](#)

Using other databases:

- [Spring JPA + SQL Server](#)
- [Spring JPA + PostgreSQL](#)
- [Spring JPA + MySQL](#)
- [Spring Data + MongoDB](#)

Part II – Swagger

Swagger editor: <https://editor.swagger.io/>

Generate Server code and client code in different languages for different platforms for the API below:

openapi: 3.0.0

info:

title: Simple API

description: A simple API for demonstration purposes

version: 1.0.0

servers:

- url: http://localhost:8000/api

paths:

/hello:

get:

summary: Returns a greeting

responses:

'200':

description: OK

content:

text/plain:

schema:

type: string

Part III – JPA OneToMany Relations

[JPA One To Many example with Hibernate and Spring Boot - BezKoder:](https://www.bezkoder.com/jpa-one-to-many/)

<https://www.bezkoder.com/jpa-one-to-many/>